FUTURE SALES PREDICTION

PROBLEM DEFINITION:

The goal of this project is to develop a predictive model that can forecast future sales of electric vehicles. This prediction will be based on historical sales data, market trends, and various influencing factors such as **Sales Rate** over a period of time, **Energy Consumption** as it works on electricity, **Fuel type** of EV vehicle, **Economic condition** such as engine type, horse power, stroke, compression ratio, wheel base, engine location, bore ratio so on.

DESIGN THINKING:

1. DATA SOURCE

Gather historical sales data for electric vehicles, including information on models, regions, time periods, and relevant market indicators. The real time driving data

- Price increase and drop
- Energy consumption
- Fuel type
- Economic conditions on EV sales.

These data are collecting from Kaggle, Related website and many other Platform.

KAGGLELINK: /kaggle/input/car-price-prediction/CarPrice_Assignment.csv

Description about the dataset used for the prediction

```
: Index(['car_ID', 'symboling', 'CarName', 'fueltype', 'aspiration',
         'doornumber', 'carbody', 'drivewheel', 'enginelocation', 'wheelbase',
         'carlength', 'carwidth', 'carheight', 'curbweight', 'enginetype',
         'cylindernumber', 'enginesize', 'fuelsystem', 'boreratio', 'stroke',
         'compressionratio', 'horsepower', 'peakrpm', 'citympg', 'highwaympg',
         'price'],
        dtype='object')
```

2. DATA PROCESSING

The data collected from various source has to be pre-processed that follows various steps such as Data cleaning to handle missing values, duplicate values, outliers and encoding categorical variables followed by data integration, data visualization and to organize the collected data to ensure it is suitable for analysis and convert the raw data into valuable one. Data processing can be processed in **Excel** or using **Python code** in Kaggle or Jupiter note book so that we would get the perfect insights of the collected data.

3. EXPLORATARY DATA ANALYSIS (EDA)

EDA makes the data look more interesting and conduct an in-depth analysis of the dataset to identify **Trends**, **Patterns** in form of histogram, box plots, scatter plots, line plots, head maps, bar charts and find the **Relationships** within the facts or **Correlations** between various factors on EV sales. The technique is often applied when the goal is to create new hypotheses or find patterns in the data. It's often used on large amounts of qualitative or quantitative data that haven't been analysed before.

4. FEATURE ENGINEERING

Create relevant features from the data that can help improve the accuracy of the predictive model. These features may include vehicle composites, electricity cost and Price Elasticity. Regardless of the data or architecture, a terrible feature will have a direct impact on the model. This technique will **equipe the data with additional features** to predict the future EV sales.

5. MODEL SELECTION

An appropriate machine learning or statistical model for sales prediction which includes time series forecasting techniques such as **ARIMA** (Auto-Regressive Integrated Moving Average) for predicting sales over-time, **Regression models** for comparing various factors, **Machine learning models** for complex relationships. Based on the complexity and learning process, learning process we would select the algorithm to fed the model to perform prediction.

6. MODEL TRAINING

Train the selected model on the historical sales data, using appropriate techniques for model validation and evaluation. Optimize model parameters for accuracy and generalization. The model is fed with various data input to check the learning process and patterns etc..Based on the model response the model parameter is changed to predict the different factors on future EV sales based on input features.

7. FUTURE DATA COLLECTION

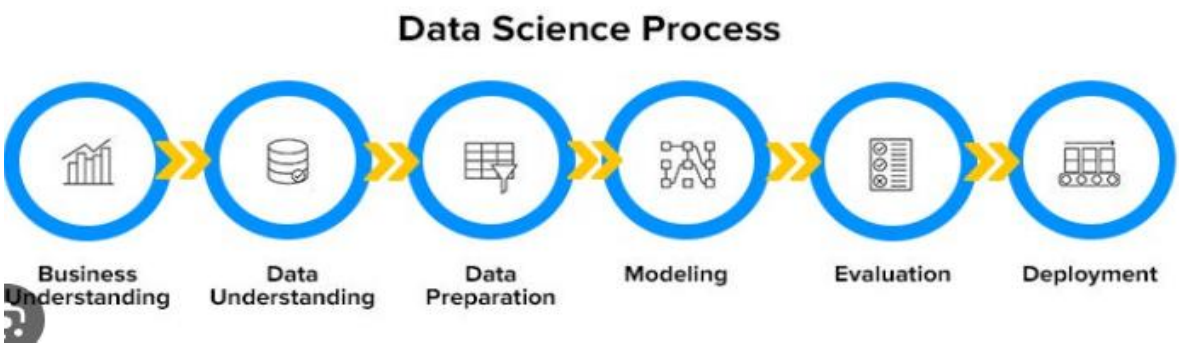
Data on factors that influence EV sales, such as changes in government policies, fluctuations in fuel prices, and advancements in battery technology collected continuously to fed the model to predict regularly.

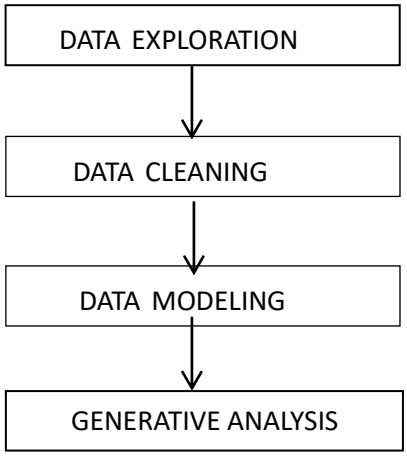
INNOVATION

The goal of the project is to develop the predictive model for predicting the future sales of electric vehicles in India. The model creation starts from collecting some past record generally known as historical data and to perform some operation to turn the data into meaningful data and obtain some insights for future need and necessity.

DESIGN STEPS

The prediction is performed in Jupiter note book using python. Dataset to perform prediction is downloaded from Kaggle.





Step1: DATA EXPLORATION

Data exploration helps to understand the overview of the data like

- Total number of rows and columns in the dataset.
- The type of variable present in every field.
- Different necessity of the field while building the model.
- To find aggregation such as sum, mean, median, standard deviation, minimum, maximum and so on.

Data exploration result whether to enhance the data by adding feature and to remove unwanted entries in the dataset.

Step2: DATA CLEANING

The next step after data exploration is data cleaning which include

- Find null values and eliminating it or add some values in place of zero
- Removing duplicate values
- Wrong numerical entry
- Check for trailing whitespace
- Check for unique values

Step3: DATA MODELING

The above step results in transformed dataset that is involved in model building and prediction. Based on the dataset the algorithm is chosen such as Linear Regression if the input and output is dependent, Logistic Regression is similar to Linear Regression but the output is discrete.

- Two variable as X and Y is selection to find its relation and it can be visualized using seaborn package in python.

- The variable is divided to test data and train data to predict the outcome using train_test_split.

The success of the model is based on the selected historical data and technique chosen for prediction.

Step4: GENERATIVE ANALYSIS

The last step involved in model building is generative analysis and where the data is tested with various input and analysis the output as Test data and Train data.

These are 4 main steps involved in any kind of prediction and there are various other steps such as

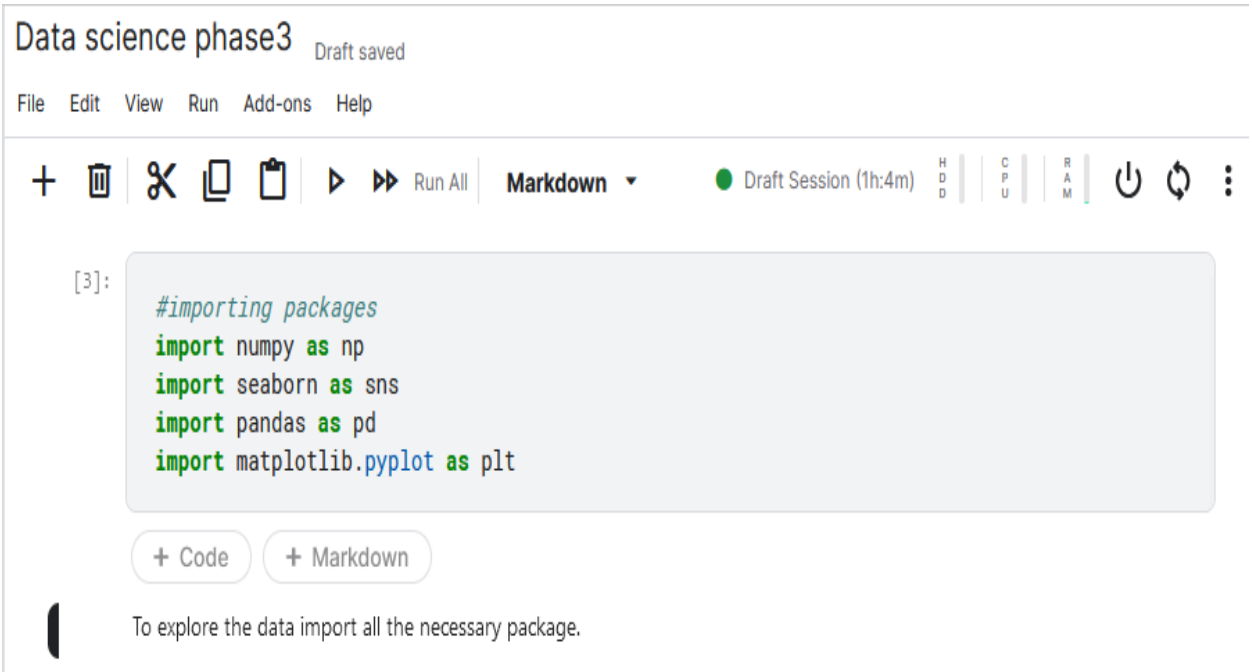
- Data preprocessing
- Exploratory data analysis
- Model selection
- Model training

The first step in this prediction is Data exploration and Data preprocessing:

The dataset used for this prediction is downloaded from Kaggle
LINK: /kaggle/input/car-price-prediction/CarPrice_Assignment.csv.

This phase includes data loading and data preprocessing using python.
These are some of the preprocessing techniques

Step1: Import all the necessary packages



Step2: Read the dataset

[5]:
```
#reading CSV file
data=pd.read_csv('/kaggle/input/car-price-prediction/CarPrice_Assignment.csv')
```

Next step is to read and load the dataset for analysis.

DATA EXPLORATION

Step 3: Data exploration using various functions

head() –Print the first 5 rows of the dataset

[8]:
```
#data analysis
data.head()
```

[8]:

ng	CarName	fueltype	aspiration	doornumber	carbody	drivewheel	enginelocation	wheelbase	...	enginesize	fuelsystem
3	alfa-romero giulia	gas	std	two	convertible	rwd	front	88.6	...	130	mpfi
3	alfa-romero stelvio	gas	std	two	convertible	rwd	front	88.6	...	130	mpfi
1	alfa-romero Quadrifoglio	gas	std	two	hatchback	rwd	front	94.5	...	152	mpfi
2	audi 100 ls	gas	std	four	sedan	fwd	front	99.8	...	109	mpfi
2	audi 100ls	gas	std	four	sedan	4wd	front	99.4	...	136	mpfi

tail()- Print the last 5 rows of the dataset

[9]:
```
data.tail()
```

[9]:

nboling	CarName	fueltype	aspiration	doornumber	carbody	drivewheel	enginelocation	wheelbase	...	enginesize	fuelsystem
-1	volvo 145e (sw)	gas	std	four	sedan	rwd	front	109.1	...	141	mpfi
-1	volvo 144ea	gas	turbo	four	sedan	rwd	front	109.1	...	141	mpfi
-1	volvo 244dl	gas	std	four	sedan	rwd	front	109.1	...	173	mpfi
-1	volvo 246	diesel	turbo	four	sedan	rwd	front	109.1	...	145	idi
-1	volvo 264gl	gas	turbo	four	sedan	rwd	front	109.1	...	141	mpfi

shape()-Print total number of rows and columns in the dataset ,i.e (columns, rows)

```
[13]: data.shape
```

```
[13]: (205, 26)
```

columns() - Print the attributes of the dataset

```
[25]: data.columns
```

```
[25]: Index(['car_ID', 'symboling', 'CarName', 'fueltype', 'aspiration',
       'doornumber', 'carbody', 'drivewheel', 'enginelocation', 'wheelbase',
       'carlength', 'carwidth', 'carheight', 'curbweight', 'enginetype',
       'cylindernumber', 'enginesize', 'fuelsystem', 'boreratio', 'stroke',
       'compressionratio', 'horsepower', 'peakrpm', 'citympg', 'highwaympg',
       'price'],
      dtype='object')
```

describe()- display the mean, standard deviation, count, mean, min,5%, 50%, 75% and max

```
[10]: data.describe()
```

[10]:

	car_ID	symboling	wheelbase	carlength	carwidth	carheight	curbweight	enginesize	boreratio	stroke	compr
count	205.000000	205.000000	205.000000	205.000000	205.000000	205.000000	205.000000	205.000000	205.000000	205.000000	
mean	103.000000	0.834146	98.756585	174.049268	65.907805	53.724878	2555.565854	126.907317	3.329756	3.255415	
std	59.322565	1.245307	6.021776	12.337289	2.145204	2.443522	520.680204	41.642693	0.270844	0.313597	
min	1.000000	-2.000000	86.600000	141.100000	60.300000	47.800000	1488.000000	61.000000	2.540000	2.070000	
25%	52.000000	0.000000	94.500000	166.300000	64.100000	52.000000	2145.000000	97.000000	3.150000	3.110000	
50%	103.000000	1.000000	97.000000	173.200000	65.500000	54.100000	2414.000000	120.000000	3.310000	3.290000	
75%	154.000000	2.000000	102.400000	183.100000	66.900000	55.500000	2935.000000	141.000000	3.580000	3.410000	
max	205.000000	3.000000	120.900000	208.100000	72.300000	59.800000	4066.000000	326.000000	3.940000	4.170000	

info()- gives general description of the dataset

```
[11]: data.info
```

```
[11]: <bound method DataFrame.info of      car_ID  symboling                   CarName fueltype aspiration  \
0         1          3        alfa-romero giulia      gas        std
1         2          3       alfa-romero stelvio      gas        std
2         3          1  alfa-romero Quadrifoglio      gas        std
3         4          2               audi 100 ls      gas        std
4         5          2                audi 100ls      gas        std
..      ...        ...                       ...      ...        ...
200     201         -1           volvo 145e (sw)      gas        std
201     202         -1               volvo 144ea      gas      turbo
202     203         -1               volvo 244dl      gas        std
203     204         -1                 volvo 246   diesel      turbo
204     205         -1               volvo 264gl      gas      turbo

    doornumber      carbody drivewheel enginelocation  wheelbase  ...  \
0          two  convertible        rwd          front       88.6  ...
1          two  convertible        rwd          front       88.6  ...
2          two    hatchback        rwd          front       94.5  ...
3         four        sedan        fwd          front       99.8  ...
4         four        sedan        4wd          front       99.4  ...
..         ...          ...        ...            ...        ...  ...
200       four        sedan        rwd          front      109.1  ...
201       four        sedan        rwd          front      109.1  ...
202       four        sedan        rwd          front      109.1  ...
203       four        sedan        rwd          front      109.1  ...
204       four        sedan        rwd          front      109.1  ...

     enginesize  fuelsystem  boreratio  stroke compressionratio horsepower  \
0           130        mpfi       3.47    2.68              9.0        111
1           130        mpfi       3.47    2.68              9.0        111
2           152        mpfi       2.68    3.47              9.0        154
3           109        mpfi       3.19    3.40             10.0        102
4           136        mpfi       3.19    3.40              8.0        115
```

We can analyse the single variable using describe() method

```
[29]: data['price'].describe()
```

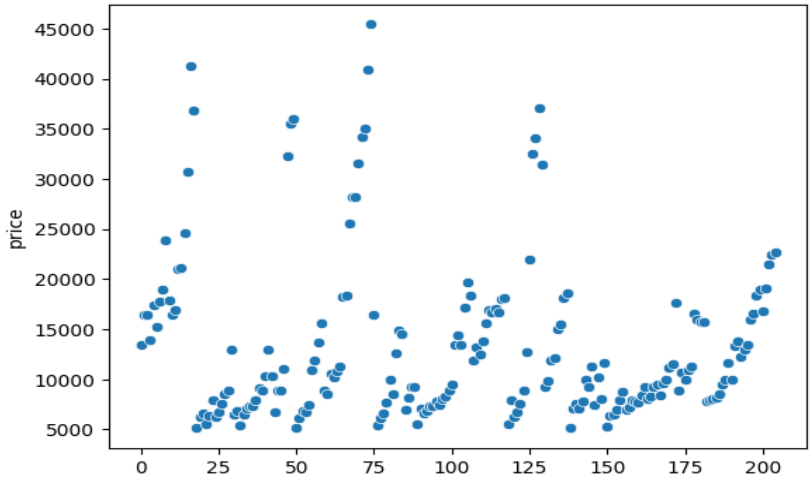
```
[29]: count      205.000000
mean     13276.710571
std       7988.852332
min       5118.000000
25%       7788.000000
50%      10295.000000
75%      16503.000000
max      45400.000000
Name: price, dtype: float64
```

VISUALIZATION

Explore the relationship between the variables using visualization

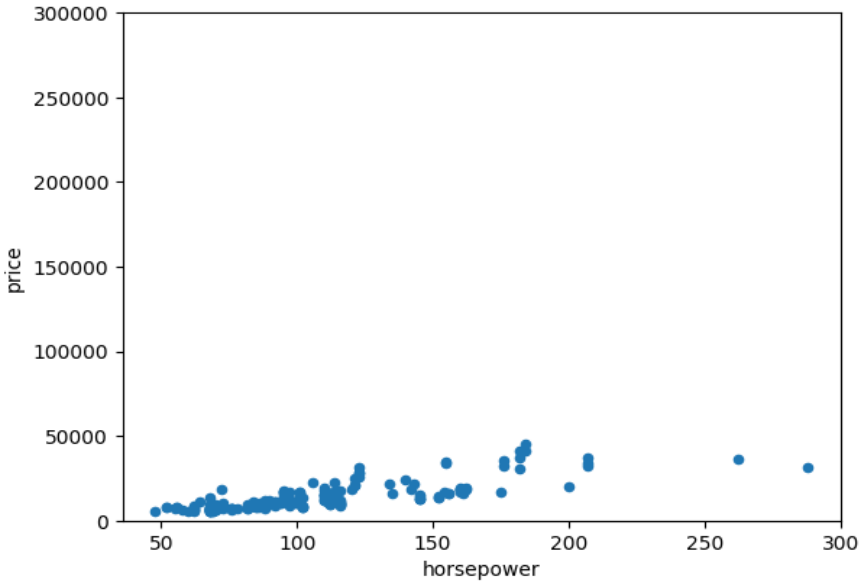
[40]:
```
sns.scatterplot(data['price'])
```

[40]: <Axes: ylabel='price'>



Using **concat()** we can combine the variables and find their relationships

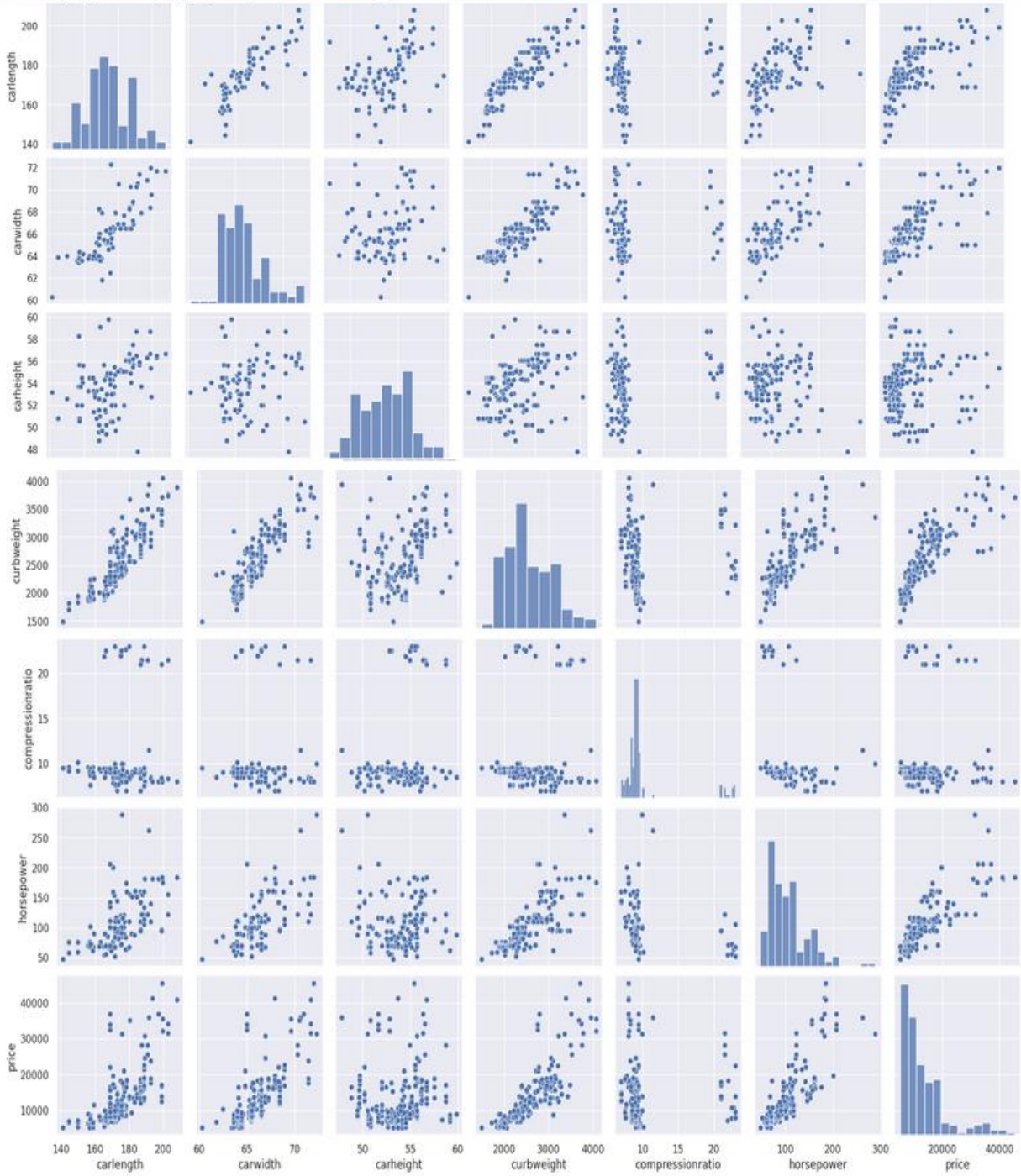
[16]:
```
var='horsepower'
df=pd.concat([data['price'],data[var]],axis=1)
df.plot.scatter(x=var,y='price',ylim=(0,300000));
```



pairplot()- plot all the defined attributes to each other attributes

```
sns.set()
cols=['CarName', 'fueltype','carlength', 'carwidth', 'carheight', 'curbweight', 'enginetype
sns.pairplot(data[cols])
plt.show()
```

```
/opt/conda/lib/python3.10/site-packages/seaborn/axisgrid.py:118: UserWarning: The figure layout has changed to
tight
  self._figure.tight_layout(*args, **kwargs)
```



DATA PREPROCESSING

In this process we check the missing values and duplicate values

```
[17]: data.isnull().sum()
```

```
[17]: car_ID              0
      symboling           0
      CarName             0
      fueltype            0
      aspiration          0
      doornumber          0
      carbody             0
      drivewheel          0
      enginelocation      0
      wheelbase           0
      carlength           0
      carwidth            0
      carheight           0
      curbweight          0
      enginetype          0
      cylindernumber      0
      enginesize          0
      fuelsystem          0
      boreratio           0
      stroke              0
      compressionratio    0
      horsepower          0
      peakrpm             0
      citympg             0
      highwaympg          0
      price               0
      dtype: int64
```

There is no missing values.

```
[38]: data.duplicated()
```

```
[38]: 0      False
      1      False
      2      False
      3      False
      4      False
             ...
      200    False
      201    False
      202    False
      203    False
      204    False
      Length: 205, dtype: bool
```

+ Code + Markdown

There is no duplicate values so it is not necessary to change the data in the dataset. If the dataset have duplicate or null values that has to be replaced if some alternate values or it should be removed.

FEATURED ENGINEERING

Featured engineering refers to manipulation of dataset to improve machine learning model training, leading to better performance and greater accuracy. The featured engineering can be performed manually and automated way.

TECNIQUE OF FEATURED ENGINEERING

- One hot encoding
- Using quartile() method
- Using mathematics and statistics such as Z-score, mean, median, mode, numbers of standard deviation.

In this project we use quartile method to feature our dataset here as follows

```
[43]: max_thresold=data['price'].quantile(0.95)
      max_thresold

[43]: 32472.399999999994
```

```
[37]: min_thresold=data['price'].quantile(0.01)
      min_thresold

[37]: 5201.12
```

To remove the outliers in sales attributes the max_threshold and min_threshold is obtained using **quartile()** by fixing some range value such as 0.95 for maximum and 0.01 for minimum.

This method is used specially when we don't have clear idea of the domain entity and understanding and one the most commonly used method in featured engineering.

```
[38]: data[data['price']< min_thresold]
```

[38]:

	car_ID	symboling	CarName	fueltype	aspiration	doornumber	carbody	drivewheel	enginelocation	wheelbase	...	enginesize	fuelsystem	bo
18	19	2	chevrolet impala	gas	std	two	hatchback	fwd	front	88.4	...	61	2bbl	
50	51	1	maxda rx3	gas	std	two	hatchback	fwd	front	93.1	...	91	2bbl	
138	139	2	subaru	gas	std	two	hatchback	fwd	front	93.7	...	97	2bbl	

3 rows × 26 columns

Using comparsion operation we detect all the outliers of min_threshold

[44]:

```
data[data['price']>max_thresold]
```

[44]:

	car_ID	symboling	CarName	fueltype	aspiration	doornumber	carbody	drivewheel	enginelocation	wheelbase	...	enginesize	fuelsystem	bo
16	17	0	bmw x5	gas	std	two	sedan	rwd	front	103.5	...	209	mpfi	
17	18	0	bmw x3	gas	std	four	sedan	rwd	front	110.0	...	209	mpfi	
48	49	0	jaguar xf	gas	std	four	sedan	rwd	front	113.0	...	258	mpfi	
49	50	0	jaguar xk	gas	std	two	sedan	rwd	front	102.0	...	326	mpfi	
71	72	-1	buick opel isuzu deluxe	gas	std	four	sedan	rwd	front	115.6	...	234	mpfi	
72	73	3	buick skylark	gas	std	two	convertible	rwd	front	96.6	...	234	mpfi	
73	74	0	buick century special	gas	std	four	sedan	rwd	front	120.9	...	308	mpfi	
74	75	1	buick regal sport coupe (turbo)	gas	std	two	hardtop	rwd	front	112.0	...	304	mpfi	
126	127	3	porcshce panamera	gas	std	two	hardtop	rwd	rear	89.5	...	194	mpfi	

Similarly we detect the outliers of max_thershold of the price attribute.

[51]:

```
da=data[(data['price']>min_thresold) &(data['price']<max_thresold)]
da
```

[51]:

	car_ID	symboling	CarName	fueltype	aspiration	doornumber	carbody	drivewheel	enginelocation	wheelbase	...	enginesize	fuelsystem
0	1	3	alfa-romero giulia	gas	std	two	convertible	rwd	front	88.6	...	130	mpfi
1	2	3	alfa-romero stelvio	gas	std	two	convertible	rwd	front	88.6	...	130	mpfi
2	3	1	alfa-romero Quadrifoglio	gas	std	two	hatchback	rwd	front	94.5	...	152	mpfi
3	4	2	audi 100 ls	gas	std	four	sedan	fwd	front	99.8	...	109	mpfi
4	5	2	audi 100ls	gas	std	four	sedan	4wd	front	99.4	...	136	mpfi
...	...	...	...	...	...	...	...	...	...	...	...	...	...
200	201	-1	volvo 145e (sw)	gas	std	four	sedan	rwd	front	109.1	...	141	mpfi
201	202	-1	volvo 144ea	gas	turbo	four	sedan	rwd	front	109.1	...	141	mpfi
202	203	-1	volvo 244dl	gas	std	four	sedan	rwd	front	109.1	...	173	mpfi
203	204	-1	volvo 246	diesel	turbo	four	sedan	rwd	front	109.1	...	145	idi
204	205	-1	volvo 264gl	gas	turbo	four	sedan	rwd	front	109.1	...	141	mpfi

191 rows × 26 columns

Using the AND comparsion operation we just exclude the outliers and obtain the reduced dataset.

```
[53]: da.shape
```

```
[53]: (191, 26)
```

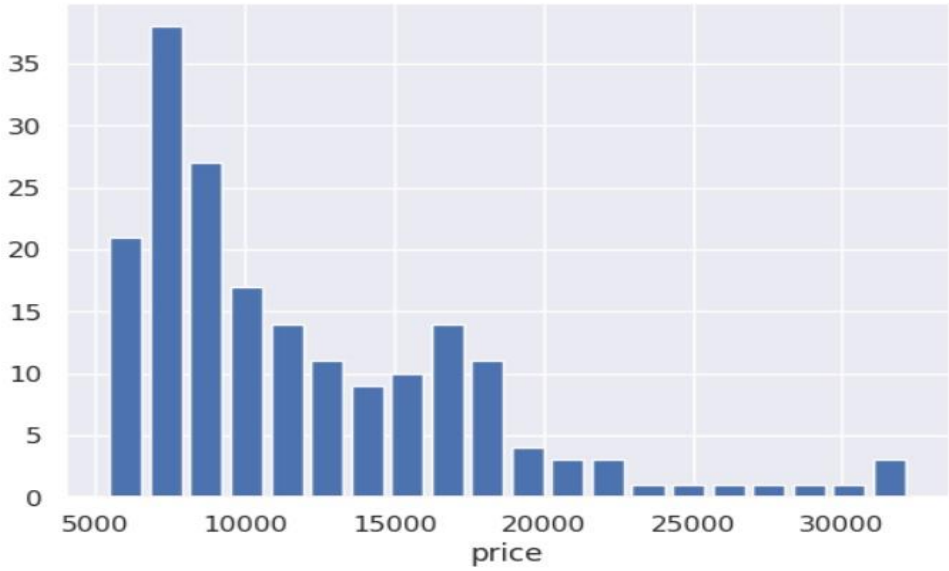
The dataset contains 191 rows and 26 columns after excluding the outliers.

Z-SCORE TECHIQUE

The z-score technique detect the standard deviation of the price attribute.

```
[56]: from scipy.stats import norm
import numpy as np
plt.hist(da.price,bins=20,rwidth=0.8)
plt.xlabel("price")
rng=np.arange(da.price.min(),da.price.max())
plt.plot(rng,norm.pdf(rng,da.price.mean(),da.price.std()))
```

```
[56]: [<matplotlib.lines.Line2D at 0x7f5006b8ead0>]
```



THREE STANDARD DEVIATION

The output of the z-score shows price range of the electric vehicle where some vehicle price is less than 5000 which is physically impossible so it shows the presence of outlier and this outliers and removed using three-standard deviation.

```
upper_limit=da.price.mean()+ 3*da.price.std()
upper_limit
```

[58]: 29016.741936097642

[59]:
```
lower_limit=da.price.mean()-3*da.price.std()
lower_limit
```

[59]: -4960.902491071465

[61]:
```
da.price.max()
```

[61]: 32250.0

The 3-standard deviation is performed and the upper-limit and the lower-limit is determined and the outlier is detected.

[63]:
```
da.price.min()
```

[63]: 5348.0

[64]:
```
da.price.mean()
```

[64]: 12027.919722513088

[65]:
```
da.price.std()
```

[65]: 5662.940737861518

Thus after reducing the outlier we find the min(), max(), mean(), std() using the method.

Now the dataset is free from outlier and the dataset is enchanced.

MACHINE LEARNING ALGORITHM used to build the model is **linear regression**

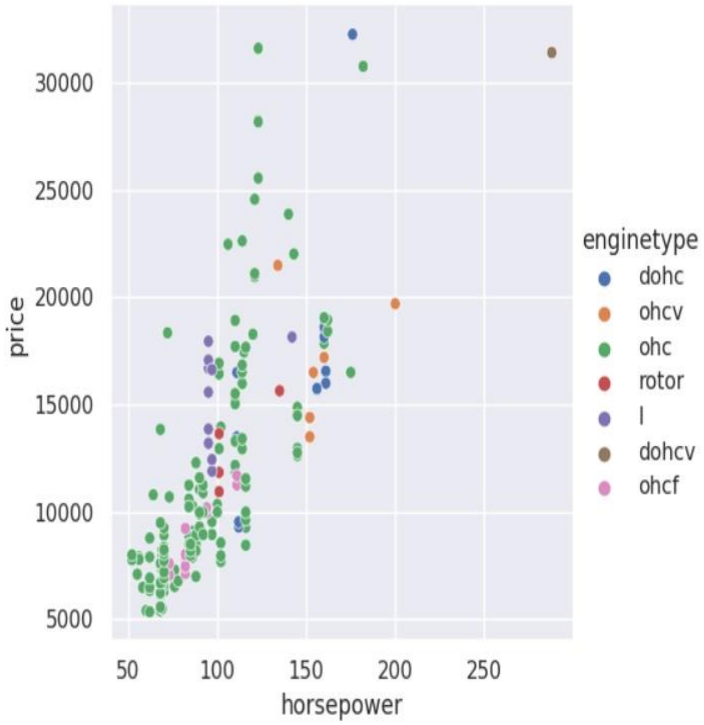
MODEL SELECTION: LinearRegression()

To train and test the data we use linear regression. Here the price act as test data and all other attributes related to price act as train data involved in prediction of price.

[40]:

```
import seaborn as sns
sns.relplot(x='horsepower',y='price',hue='enginetype',data=da)
```

[40]: <seaborn.axisgrid.FacetGrid at 0x7dad6c588eb0>



Import the sklearn and train_test_split package to train the data

[44]:

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

Steps involved in training and testing the data

Declare the train and test attributes separately

```
train = da.drop(['car_ID','symboling','CarName' ,'fueltype','aspiration' , 'doornumber'  ,'carbody','drivew
 'curbweight','enginetype','cylindernumber','enginesize' ,'fuelsystem' ,'horsepower','peakrpm','citympg','h
test = da['price']
```

+ Code + Markdown

[75]:
```
x_train, x_test, y_train, y_test= train_test_split(train, test, test_size=0.25,random_state=2)
```

[78]:
```
model =LinearRegression()
```

Declare the LinearRegression() to the variable name model

[79]:
```
model.fit(x_train,y_train)
```

[79]:
```
▾ LinearRegression
LinearRegression()
```

The train and test data is fit to the model. A good model the prediction result is 1.0, negative result is the worst model and between values the model can be improved by enhancing the dataset. The data that are inconsistent can be changed and the model can be enchanced and improved that predict the accurate result.

```
[80]: final=model.predict(x_test)
```

```
[81]: final
```

```
[81]: array([ 6676.07270484, 14516.04190322,  9910.43476094, 10444.75866976,
        8004.86173136,  9728.25099958, 12250.30384207,  6375.08732555,
       10554.87017902, 12810.28614734, 12935.6515465 , 12490.4861257 ,
        9710.61247727, 14403.40717528, 18635.59713355, 12360.60868962,
       23305.69209438,  8205.83225148, 14420.83152347, 17617.95081719,
       13718.54164717,  8046.55862517,  7418.31934869, 14477.53735535,
       10608.12755964,  7902.84216705, 15942.52811759, 20522.81717191,
        7223.33012156,  9710.61247727, 16100.59515354,  6650.99962501,
       12490.4861257 ,  8672.74670638, 14535.78546907,  8283.39672714,
       12661.59724236, 12661.59724236, 18504.09613651, 13862.21272389,
       13426.11263276,  6177.21962079, 25396.65945057,  6628.34071608,
       12661.03289781, 13492.88392869, 14536.44358793, 10444.75866976])
```

The prediction of each row attribute is stored in final variable

```
[82]: model.score(x_test,y_test)
```

```
[82]: 0.47228360470730635
```

Finally the prediction score of the developed model is 0.47228 that can be be improved for better prediction by enhancing the dataset.

CONCLUSION

The aim is to build a well designed and trained predictive model that significantly impact the accuracy and reliability of sales forecast. The design and its steps plays a crucial role in future sales prediction. A thoughtful approach to feature engineering, model selection, data preprocessing, and ongoing monitoring ensures that the model remains valuable and aligned with the goals of the business since it is the fast evolving technique in recent days all over the world and it is a great challenge to the electric vehicle manufacturing company to monitor needs and necessary. So this project helps to predict various factors involving EV and convert the raw data into valuable insights that can be used for future development.